

Váci SzC Petőfi Sándor Műszaki Technikum, Gimnázium és Kollégium

Záródolgozat

SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ

One Knight Army



Készítette: Juhász Márk Ferenc 13.D

Csapattársak: Bangó Máté, Bacskai Bence

Konzulens: Berdó Tamás

Aszód

2026

Nyilatkozat

Alulírott **Juhász Márk Ferenc** tanuló kijelentem, hogy a szakdolgozat teljes mértékben a saját munkám eredménye. A felhasznált eszközöket és szakirodalmakat (tanulmány, internetes forrás, személyes közlés stb.) idézőjel és megjelölt források nélkül nem építettem be, egyértelműen megjelöltem. Az elkészült záródolgozatban található eredményeket a képzőintézmény és a feladatot kiíró intézmény saját céljára térítés nélkül felhasználhatja, a titkosításra vonatkozó esetleges megkötések mellett.

Aszód,.....

.....

Juhász Márk Ferenc

KONZULTÁCIÓS LAP

Szoftverfejlesztő és tesztelő

A tanuló neve: Julian Kévé Senere Osztálya: 13.D

A záródolgozat témája: Unity motorban játékfejlesztés
a One Bright tananyag című játék megvalósítása

Sorszám:	A konzultáció időpontja:	A konzulens aláírása:
1.	2026. jan. 18	Boró Tamás
2.	2026. febr. 22	Boró Tamás
3.	2026. ápr. 03.	Boró Tamás
4.		
5.		

A záródolgozat beadható.
Aszód,

Boró Tamás
konzulens/

Tartalomjegyzék

1.	Bevezetés	7
2.	Felhasználói dokumentáció	8
2.1	Rendszerkövetelmények	8
2.1.1	Hardver követelmények	8
2.1.2	Szoftver követelmények	8
2.2	A program telepítése	9
2.3	A játék használatának részletes leírása	10
2.3.1	Főmenü	10
2.3.2	Beállítások	12
2.3.3	Karakter irányítása	14
2.3.4	Harcrendszer és harcmódok	15
2.3.5	Kereskedelmi rendszer, Shop Menü	16
2.3.6	HUD – Játékmenet közbeni állapotjelzők	18
2.3.7	Karakterstatisztika menü (23. ábra)	20
2.3.8	Képességfa (24. ábra)	21
2.3.9	Szüneteltetés (pause) menü	23
2.3.10	A Wave-rendszer (Hullámok) működése	24
2.3.11	Ellenségek és tulajdonságaik	25
3.	Fejlesztői Dokumentáció	27
3.1	Témaválasztás	27
3.2	Alkalmazott fejlesztői eszközök	28
3.2.1	Programok, szoftverek, alkalmazások	28
3.2.2	A Unity beépített rendszerei	29
3.2.3	A Unity beépítette rendszerei ÁBRÁK	30
3.3	Adatmodell leírása	31
3.3.1	ScriptableObject alapú adatmodell (ItemSO)	31
3.3.2	Dinamikus adatszerkezetek	31
3.3.3	Osztályhierarchia és öröklődés	32
3.4	Részletes feladat-specifikáció, megvalósítás	33
3.4.1	Főmenü (Main Menu) technikai megvalósítása	33
3.4.2	Inventory és tárgykezelő rendszer	35
3.4.3	Inventory Ábrák	37
3.4.4	Kereskedelmi rendszer (Shop) technika felépítése	38

3.4.5	Játékmenet és ellenségkezelés	41
3.4.6	Vár élete és újraéledési rendszer	42
3.4.7	Rendszerszintű vezérlők	43
3.4.8	Vizuális effekt és optimalizálás	44
3.4.9	Tesztesetek	45
3.4.10	Hibák felsorolása (bugok)	46
3.4.11	Továbbfejlesztési lehetőségek	47
4.	Összegzés	48
5.	Ábrajegyzék	49
6.	Irodalomjegyzék, forrásmegjelölés	51

1. Bevezetés

A szakdolgozatom témája a One Knight Army, egy felülnézetes (top-down), 2 dimenziós akció-RPG videójáték. A projekt célja egy olyan magával ragadó játékelmény létrehozása, amely a folyamatos fejlődési lehetőségek és a taktikus védekezés ötvözésével fenntartja a játékos érdeklődését a kezdetektől egészen a végkifejletig.

A játék menete

A játék története szerint a játékos egy magányos lovagot alakít, akinek egyetlen feladata, a vár védelme az erdő mélyéről előtörő szörnyek hordáitól. A játékmenet 15, egyre nehezedő hullámból áll, melyek végén a játékosnak egy hatalmas Ogre főellenséggel (boss) kell szembenéznie a győzelem elérése érdekében.

Csapatmunka és Feladatmegosztás

A projektet három fős csapatban készítettük: Bangó Mátéval és Bacskai Bencével. A közös munka baráti légkörben, rendkívül gördülékenyen és hatékonyan zajlott. A fejlesztési feladatokat az alábbiak szerint osztottuk meg:

- Személyes hozzájárulásom: Én készítettem a játékpályát, a különféle ellenségeket, a hullámok működését, a boltot, a főmenüt és beállításokat, az inventory rendszert.
- Bacskai Bence: Ő volt felelős a karakter és az ellenségek mozgásáért, támadásért, elkészítette a talentumfát, a szintlépési rendszert.
- Bangó Máté: Ő felelt a játék hangjaiért, valamint a játék 2 nyelvű szövegéért. Ezek mellett csinált egy weboldalt ahol a játék letöltése van kifejtve.

2. Felhasználói dokumentáció

2.1 Rendszerkövetelmények

2.1.1 Hardver követelmények

	Minimális	Ajánlott
Memória	4GB	8GB
Videókártya	Integrált (Intel HD 5000)	Nvidia GTX 960
Processzor	Intel i3 4000	Intel i3 6000
Kijelző felbontás	1280x720	1920x1080
Tárhely	1GB	5GB

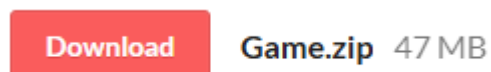
2.1.2 Szoftver követelmények

Specifikáció	Minimális verzió
Direct X	11/12
Operációs rendszer	Windows 10 64bit/+
Visual C++ Redistributable	✓

2.2 A program telepítése

- A játék letöltése:
 - <https://bacsek.itch.io/oneknightarmy>
 - Az alábbi linket megnyitva, találkozunk a projekt hivatalos letöltő oldalával. A „Download” gomb rákattintásával tölthetjük le a játékot. Letöltéskor egy „Game.zip” fájlal fogunk találkozni, melyet ki kell csomagolni.

Download



- A fájl kicsomagolásához egy archiváló programra lesz szükségünk. A tömörített mappa kicsomagolása után kétszer kattintsunk a benne lévő, „One Knight Army.exe” fájlra, mellyel a játék elindul.

	D3D12	2026. 04. 15. 15:31	Fájlmappa	
	MonoBleedingEdge	2026. 04. 15. 15:31	Fájlmappa	
	TempUnity_Data	2026. 04. 15. 15:31	Fájlmappa	
	One Knight Army	2026. 04. 15. 15:31	Alkalmazás	652 KB
	UnityCrashHandler64	2026. 04. 15. 15:31	Alkalmazás	1 572 KB
	UnityPlayer.dll	2026. 04. 15. 15:31	Alkalmazáskiterjes...	34 958 KB

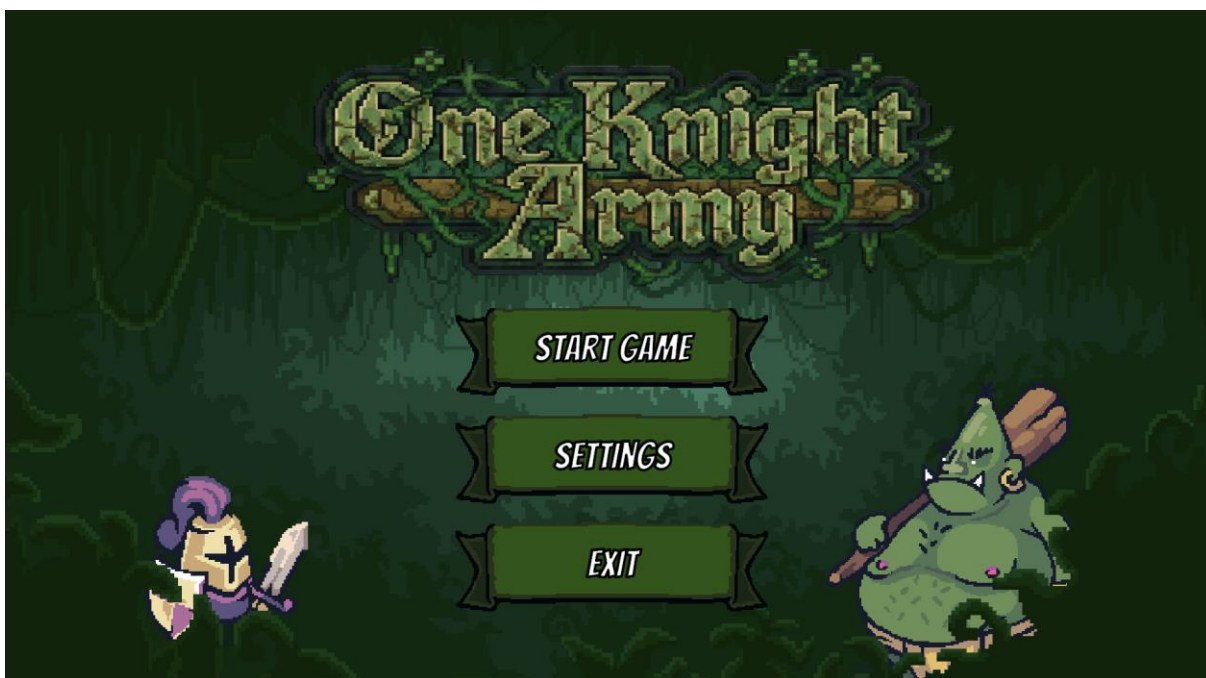
2.3 A játék használatának részletes leírása

2.3.1 Főmenü

A szoftver elindítását követően a felhasználó a Főmenübe (Main Menu) érkezik. Ez a felület szolgál a játék indítására, a beállítások testreszabására és a programból való kilépésre. A menü megjeleníti a főszereplő lovagot és a végső ellenséget, az Ogrét is.

A Főmenü vizuális elemei:

- A játék logója: A képernyő felső részén, és a 2. ábrán látható indás logó.
- A főhős: A képernyő bal oldalán látható lila lovag.
- A főellenség: A képernyő jobb oldalán látható zöld szörny.



1. ábra: A One Knight Army főmenüje



2. ábra: A One Knight Army logója

A Főmenü elemei:

- Start Game: Erre a gombra kattintva a játékos belép a játékpályára, melyen elkezdheti a játékot.



3. ábra: Start Game gomb

- Settings: Erre a gombra kattintva felugrik a beállítások menü, melyen a játékos tudja módosítani a játék beállításait.



4. ábra: Settings gomb

- Exit: Erre a gombra kattintva a program bezárul.



5. ábra: Exit gomb

A főmenü vizuális elemei „életre keltése”:

- Logó: A játék elindításakor a játék logó először becsúszik fentről, majd fel és le mozog.
- Karakterek: A játék elindításakor a karakterek becsúsznak lentől, majd egy alap mozgási animációt játszanak le.
- Gombok: A játék elindításakor a gombok először átlátszóak, majd egy idő után lassan megjelennek.
- A főmenü háttere több képből áll, és a kurzor mozgatásával a különféle képek különbözően mozognak, ezzel egy „parrallax” hatást alkotnak.

2.3.2 Beállítások

A főmenün lévő, 4. ábrán látható „SETTINGS” gomb rányomása után, a 6. ábrán látható beállítások menü jelenik meg. A beállítások menüpont célja, hogy a játékos a saját igényeihez és számítógépes teljesítményéhez igazíthassa a játék futását.



6. ábra: Settings menü

Graphics Quality (Grafikai minőség): A gombra rányomva megjelenik a 7. ábrán látható legördülő menü, melyben találhatóak a grafikai opciók. Itt állítható be a játék vizuális részletessége. Az „ULTRA” beállítás a legmagasabb minőséget képviseli, még a „LOW” a legalacsonyabbat. Ez az opció lehetővé teszi, hogy a gyengébb erőforrásokkal rendelkező számítógépeken is élvezhetően fusson a játék a grafikai részletesség csökkentésével.



7. ábra: Grafikai menü

Hangerő szabályozók: A játék hangrendszere **három** különálló csatornára van bontva:

- **Master Volume (Globális hangerőszabályozó):** A játék összes hangjának globális erejét szabályozza.
- **Music Volume (Zene hangerőszabályozó):** Kizárólag a háttérzene hangerejéért felel.
- **SFX Volume (Hangeffekt hangerőszabályozó):** A hangeffektek erejét állítja.

A hangerőszabályozókon lévő zöld nyíl húzásával lehet állítani a hangerőt, bal oldali irányban csökken a hangerő, jobb oldali irányban pedig növekszik.

Exit (Vissza) gomb: A gombra kattintva a játékos visszatér az előző képernyőre.

2.3.3 Karakter irányítása

A One Knight Army irányítása a legtöbb PC-s játéknál megszokott, intuitív sémát követi. A játékos a lovagot billentyűzet segítségével, vagy controllerrel is tudja irányítani. A karakter mozgása nyolc irányban lehetséges (függőleges, vízszintes és átlós irányok)

Billentyűzetes mozgás:

• <i>Klasszikus WASD</i>	• <i>NYÍLBILLENTYŰK</i>	• <i>IRÁNY</i>
• W	• ↑	• <i>fel</i>
• A	• ←	• <i>bal</i>
• S	• ↓	• <i>le</i>
• D	• →	• <i>jobb</i>

8. ábra: Karakter mozgása



9. ábra: Lovag futás

2.3.4 Harcrendszer és harcmódok

A One Knight Army harcrendszere a közelharcra, távolsági támadásokra és a védekezésre épül. A játék kezdetén a lovag csak az alapvető csapással rendelkezik, azonban a fejlődés során új harci módok válnak elérhetővé.

Alapvető támadás: (10.ábra)

- Space billentyű: A lovag suhint a kardjával a nézési irányába.

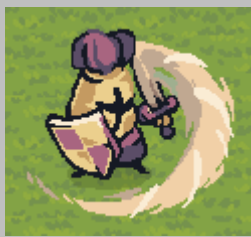
Fejleszthető képességek:

➤ Íjászat (11.ábra)

- Q billentyű: Váltás a kard és az íj között.
- Space billentyű: A lovag egy nyílvesszőt lő ki.

➤ Védekezés (12.ábra)

- Shift billentyű: A lovag előveszi a pajzsát, mellyel egy ütést le tud védeni az ellenségtől.



10. ábra: Lovag csapás



12. ábra: Lovag védekezés



11. ábra: Lovag íjászat és lövés

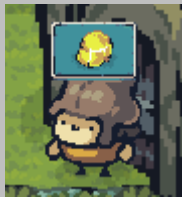
2.3.5 Kereskedelmi rendszer, Shop Menü

A játékos a megszerzett aranyat a vár területén található kereskedő (Shopkeeper) NPC-nél tudja elkölteni. (13.ábra)

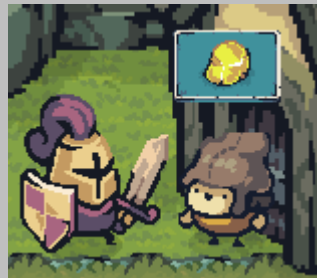
Interakció a kereskedővel

A fejlesztés során fontos szempont volt, hogy a játékos egyértelmű visszajelzést kapjon arról, hogy mikor kereskedhet a boltossal.

- Érzékelési zóna: A kereskedő rendelkezik egy láthatatlan zónával, amint a lovag belép ebbe a zónába, kereskedhet az NPC-vel.
- Vizuális visszajelzés: Amint belép a zónába a játékos, az NPC feletti ikon fel-le ugráló animációval jelzi, hogy készen áll az interakcióra. (14.ábra)
- Megnyitás: Amíg a játékos a zónán belül tartózkodik, az „E” billentyű megnyomásával megnyithatja a bolt felületét. (15.ábra)



13. ábra: Kereskedő NPC



14. ábra: Ikon visszajelzés

A bolt kezelőfelülete (15. ábra)

A bolt megnyitásakor a játékidő megáll, biztosítva, hogy a játékost ne érje támadás vásárlás közben. A bolt kezelőfelület bal oldalán megjelenik a kereskedő, aki egy kedves üzenettel köszönt.

A bolt három féle létfontosságú főzetet (potion) kínál:

- Heal Potion:
 - Ár: 30 arany
 - A játékosnak egyből vissza ad 20 életerőt.
- Swiftess Potion:
 - Ár: 60 arany
 - A játékos mozgási sebességét növeli 20 másodpercig.
- Extra HP Potion:
 - Ár: 50 arany
 - A játékos maximális életerejét növeli 30 életerővel, 20 másodpercig.



15. ábra: Bolt felület

2.3.6 HUD – Játékmenet közbeni állapotjelzők

A játékmenet során a képernyőn folyamatosan látható egy információs felület (HUD), amely a játékos túléléséhez és fejlődéséhez szükséges legfontosabb adatokat jeleníti meg valós időben.

Életerő mutatók

- **Lovag életerő (16. ábra)**
 - A karakter pontos aktuális életerejét látja a játékos, így tudja ha gyógyításra van szüksége
- **Bázis életerő (17. ábra)**
 - A játékos látja, a vár pontos életerejét 5 egységnyi életerővel rendelkezik, melynek elvesztése a játék végét jelenti.

Gazdaság és Inventory

- **Arany (18. ábra)**
 - A szörnyek legyőzéseért járó fizetőeszköz. A számláló mutatja az aktuálisan elkölthető összeget, amelyet a boltban használhatunk fel.
- **Inventory (19. ábra)**
 - Itt láthatóak a játékosnál lévő főzetek, a játékos látja, hogy melyik típusból hány darab áll rendelkezésre. (20. ábra)
 - A főzeteket a játékos a bal egér gomb kattintásával tudja elhasználni.
- **XP és Szintjelző (21. ábra)**
 - A szörnyek legyőzése után kapott tapasztalati pontok (XP) egy sávon töltődnek, amint betöltődött a sáv, a játékos szintet lép, amit elkölthet a Skill Tree-ben.

Idő alapú visszajelzők (22. ábra)

- Amikor a játékos elfogyaszt egy speciális főzetet, vagy elhasznál egy képességet, megjelenik egy fogyó állapotjelző sáv.
- Működése: A csúszka hatóidő kezdetekor teljesen feltöltött állapotban kezd, majd folyamatosan csökken. Amint a sáv kiürül, az erősítés megszűnik.

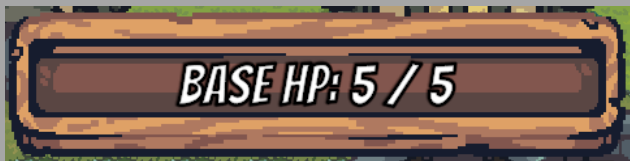
HUD Ábrák



16. ábra: Lovag életerő



18. ábra: Arany számláló



17. ábra: Vár életerő



19. ábra: Inventory



20. ábra: Inventory feltöltve



21. ábra: Tapasztalati pont



22. ábra: Időalapú visszajelzők

2.3.7 Karakterstatisztika menü (23. ábra)

A játékos aktuális erejét reprezentáló felület a „T” billentyű megnyomásával hívható elő. Megnyitásakor a játékidő megáll, hogy a játékost ne érje támadás közben. A menü központi eleme a lovag portréja. A felület alsó részén a szintjelző jelenik meg nagyobb méretben, hogy a játékos pontosabban lássa a tapasztalati pontjait.

Statisztikák hatása a játékmenetre:

- Strength (Erő): Ez az érték határozza meg a lovag közvetlen sebzését.
- Agility (Gyorsaság): Ez az érték határozza meg a lovag mozgási sebességét.
- Vitality (Életerő): Ez az érték határozza meg a lovag maximális életerejét. A rendszer a maximális életerő / 10 képlet alapján számolja ki az értéket.



23. ábra: Karakterstatisztika menü

2.3.8 Képességfa (24. ábra)

Navigáció a képességfához:

- A karakterstatisztika menüben lévő lovag portréja nem csak egy dizájn elem, hanem egy gomb is, amivel a játékos megtudja nyitni a képességfát.

A One Knight Army izgalmasságát a Képességfa rendszer adja, amely lehetővé teszi, hogy a játékos a saját stílusához igazítsa a lovag tulajdonságait. A szintlépések során gyűjtött pontok itt válthatóak be fejlesztésekre.



24. ábra: Képességfa

A választható képességek leírása:

- **Sebzés:** A karakter alapsebzését növeli meg állandó jelleggel (maximum 10 szint)
- **Maximális életerő:** A lovag maximális életerejét növeli, lehetővé téve a hosszabb ideig tartó harcot (maximum 10 szint)
- **Gyorsaság:** A lovag mozgási sebességét növeli, segítve a gyors pozícióváltást és a kitérést (maximum 5 szint)
- **Nyíl zápor:** Fejlett íjász képesség. Csökkenti a lövések közötti várakozási időt, és növeli a nyilak sebzését (maximum 5 szint)
- **Íjzás feloldás:** Ennek feloldása után válik elérhetővé az íj használata a játékban.
- **Pajzs feloldás:** Feloldja a pajzs használatát, mellyel a lovag képes lesz védekezni.
- **Gyógyítás képesség:** Egy öngyógyító képesség. Minél magasabb a szintje, annál több életerőt képes a lovag visszanyerni a használatával.
- **Düh képesség:** A játék legerősebb képessége, mellyel a lovag halhatatlan lesz 3 másodpercig. A képesség feloldása 5 pontot igényel.

Képességek elhasználása:

- A játékos a képességeket a bal egér gomb kattintásával tudja elhasználni, a képességeket az inventory bal és jobb oldalán találja.

Sebzés	Maximális Életerő	Gyorsaság	Nyíl zápor
			
Íjzás feloldás	Pajzs feloldás	Gyógyítás képesség	Düh képesség
			

2.3.9 Szüneteltetés (pause) menü

A játék bármely szakaszában az „ESC” billentyű megnyomásával előhívható a Szünet menü. A menü meghívásával megáll a játékidő, hogy a játékost ne érje támadás közben (25. ábra).

A szünet menü opciói:

- Folytatás (Resume Game): Bezárja a szüneteltető felületet, és visszatér a normál játékmenetre.
- Beállítások (Settings): Megnyitja a korábban részletezett konfigurációs panelt (grafika, hangerő), anélkül, hogy a játékosnak ki kellene lépnie a főmenübe.
- Vissza a főmenübe (Main Menu): Megszakítja az aktuális játékmenetet, és visszairányítja a felhasználót a főmenübe.



25. ábra: Szünet menü

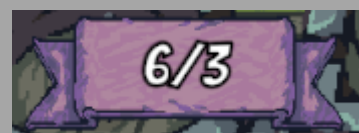
2.3.10 A Wave-rendszer (Hullámok) működése

A játékmenet ciklikus felépítés, ahol a játékosnak egyre nehezedő támadási hullámokat kell túlélnie.

- **Felkészülési idő:** A játék nem automatikusan indítja el az ellenségeket. Ez lehetőséget ad a játékosnak, hogy a harcok között ellátogasson a boltba, pontokat osszon el a képességfán, vagy stratégiaileg pozicionálja magát. (26. ábra).
- **Hullám indítása:** A Képernyő tetején található „Start The Wave” gombbal indítható el. (27. ábra).
- **Hullám indikátor:** A hullám elindítása után, előjön ez az indikátor, ami mutatja, hogy a játékos éppen hanyadik hullámnál tart. (28. ábra).
- **Ellenség számláló:** A hullám elindítása után, megjelenik a számláló, mely mutatja, hogy az adott hullámban mennyi ellenség lesz összesen és, hogy mennyi ellenség van éppen a pályán. (29. ábra).
- **Hullám vége:** Egy hullám akkor ér véget, ha az összes ellenséget legyőzte a játékos a hullámból. Ekkor a gomb ismét megjelenik, és a játékos készülhet a következő fázisra.
- **Progresszió:** Összesen 15 hullám található a játékban. A nehézségi görbe emelkedik: az ellenségek száma nő, életerejük és sebzésük emelkedik, illetve új típusú szörnyek jelennek meg.



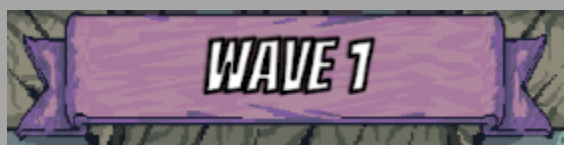
26. ábra: Felkészülés



29. ábra: Ellenség számláló



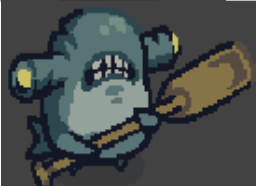
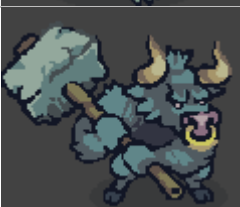
27. ábra: Hullám indító gomb



28. ábra: Hullám indikátor

2.3.11 Ellenségek és tulajdonságaik

A játékban különböző viselkedési mintával rendelkező ellenségek találhatók. Közös tulajdonságuk, hogy alapértelmezetten a játékost támadják, de ha nincs a játékos a látómezőjükben, akkor a várkapu támadására fókuszálnak.

Ellenség	Kinézet	Életerő	Sebzés	Arany	XP
Csontváz		20	5	5	8
Lapátos Hal		35	8	5	15
Pók		60	20	10	20
Panda		100	30	20	40
Medve		300	30	100	100
Minotaurusz		600	45	200	200
Teknős		1000	30	300	300

A végső kihívás: Az Ogre (főellenség) (29. ábra)

A 15. hullám végén megjelenik a játék főellensége, az Ogre. Ő az egyetlen olyan ellenség, amely nem csak a játékosra, hanem közvetlenül a várkapura is jelent végzetes veszélyt, mivel ha eléri a várkaput akkor egyetlen csapással véget vet a játéknak. Legyőzése a játék megnyerését jelenti. A hagyományos ellenségekkel ellentétben összetettebb harci taktikát alkalmaz a játékos ellen. Az Ogre egy támadással kettőt is suhint, ezért kétszer annyit sebez.

Az Ogre tulajdonságai:

- Életerő: 1500
- Sebzés: 60 (Egyetlen suhintás)

Különleges képessége: Rohamozás (30. ábra)

Az Ogre egy dinamikus támadási formával rendelkezik, amely megnehezíti a játékos számára a távolságtartást:

- **Aktiválási feltétel:** Amint a játékos egy bizonyos közelségbe ér az Ogréhez, az ellenség aktiválja a képességét. Ekkor hirtelen megnövelt sebességgel megindul a lovag irányába.
- **Kombinált támadás:** A roham nem csak helyváltoztatás, a képesség közben üti a játékos.
- **Újratöltési idő:** A játékegyensúly érdekében a képesség nem használható folyamatosan. A roham után egy ideig nem tudja újra használni a képességet.



29. ábra: Ogre, végső ellenség



30. ábra: Ogre képessége, Rohamozás

3. Fejlesztői Dokumentáció

3.1 Témaválasztás

A szakdolgozatom tárgya a One Knight Army nevű, felülnézetes (top-down), kétdimenziós akció-RPG szoftver fejlesztése. A projekt választását elsősorban a komplex játékmechanikák és a szoftverarchitektúra tervezésének kihívásai indokolták.

Fejlesztői célkitűzések:

- Objektumorientált szemlélet: Olyan moduláris kód felépítése, amely lehetővé teszi az ellenségek, képességek és statisztikák könnyű bővíthetőségét.
- Mesterséges Intelligencia (AI): Különböző viselkedési minták (pathfinding, aggro-távolság, speciális boss-mechanikák) leprogramozása.
- Felhasználói élmény (UX/UI): Egy összetett, mégis átlátható kezelőfelület és fejlődési rendszer (Skill Tree) implementálása, amely valós időben kommunikál a játéklogikával.

A projekt technikai alapvetései:

A szoftver egy 15 hullámból álló, eseményvezérelt rendszert valósít meg. A technikai kihívást a különböző entitások (játékos, ellenségek, várkapu) közötti interakciók, a tárgykezelés (Shop), valamint a szekvenciális hullám-logika pontos vezérlése jelentette. A projekt végkifejlete egy összetett állapotgéppel rendelkező főellenség (Ogre) harci mechanikájának kidolgozása volt.

3.2 Alkalmazott fejlesztői eszközök

3.2.1 Programok, szoftverek, alkalmazások

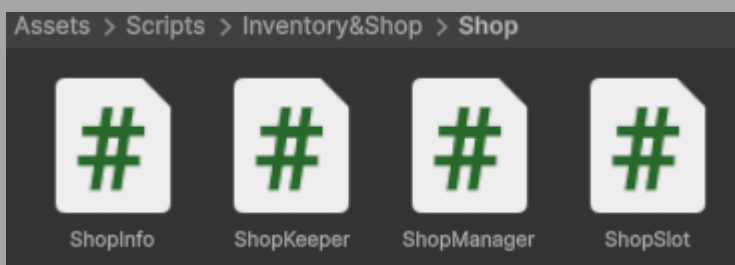
A projekt megvalósítása során modern, iparági sztenderdek megfelelő eszközöket alkalmaztam, amelyek biztosították a hatékony kódolást és a vizuális elemek integrációját.

- Unity Game Engine:
 - A játék alapjául szolgáló keretrendszer. Azért esett a választásom a Unity-re, mert kiemelkedő támogatást nyújt a 2D-s projektekhez, rugalmas a komponens-alapú rendszere, és széleskörű dokumentációval rendelkezik a C# alapú fejlesztéshez.
- Microsoft Visual Studio (2022):
 - A C# kódok írásához, hibakereséshez és az IntelliSense (automatikus kódkiegészítés) funkciók kihasználásához használtam, ami jelentősen gyorsította a fejlesztési folyamatot.
- Aseprite / Piskel:
 - A játék grafikai elemeinek, a pixel art stílusú karaktereknek, animációknak és a GUI (felhasználói felület) elemeinek elkészítéséhez használt képszerkesztő szoftver.
- Git / GitHub:
 - Verziókezelő rendszer, amely segített a forráskód mentésében, a változtatások nyomon követésében.

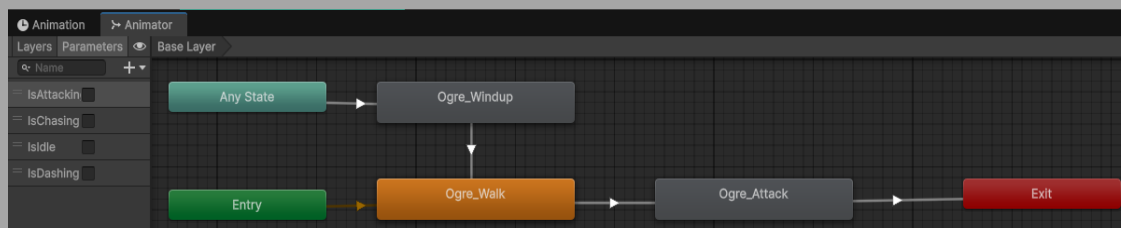
3.2.2 A Unity beépített rendszerei

- Unity Component-Based Architecture (31. ábra):
 - A szoftver tervezésekor a komponens-alapú szemléletet követtem. Ez lehetővé tette, hogy az egyes játékelemeket (pl. Shop) független szkriptekre bontsam, amelyek saját feladatkörrel rendelkeznek (pl. Shopinfo, ShopKeeper, ShopManager, ShopSlot).
- Animation State Machine (Animator) (32. ábra):
 - A karakterek vizuális állapotait (idle, run, attack,) egy logikai állapotgép vezérli. A szkriptekből átadott paraméterek (pl. isWalking bool, attack trigger) biztosítják a folyamatos és reszponzív animációkat.
- UI Canvas System (33. ábra):
 - A játékfelület (HUD, Shop, Skill Tree) a Unity Canvas rendszerére épül. A dinamikus elemek (HP csíkok, Cooldown sliderek) valós időben frissülnek a logikai változók értékei alapján.
- Time Management (Időkezelés) (34. ábra):
 - A Unity beépített Time osztályát használtam a játékmenet szüneteltetésére (timeScale), valamint a keretfüggetlen mozgások kiszámításához (deltaTime), biztosítva, hogy a játék sebessége minden hardveren azonos legyen.
- Prefab System (Előregyártott objektumok) (35. ábra):
 - A szoftverarchitektúra egyik alapköve. A szörnyeket, lövedékeket és gyűjthető tárgyakat Prefabként mentettem el, így a Spawn folyamat során (hullámok indítása) a motor dinamikusan, futásidőben tudja példányosítani (Instantiate) őket a memóriába.
- Tilemap Renderer (36. ábra):
 - A játéktér és a vár környezetének hatékony felépítéséhez a Tilemap rendszert alkalmaztam. Ez lehetővé tette a pályaelemek (fű, falak, díszítőelemek) rétegenkénti elhelyezését és a statikus ütközők (Tilemap Collider 2D) gyors regenerálását.

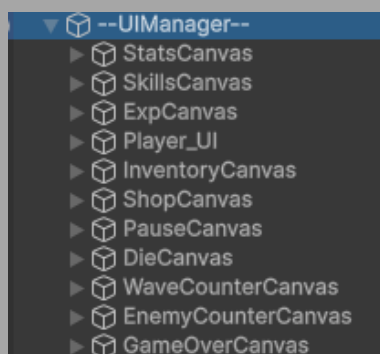
3.2.3 A Unity beépítette rendszerei ÁBRÁK



31. ábra: Shop System Feladatkörei



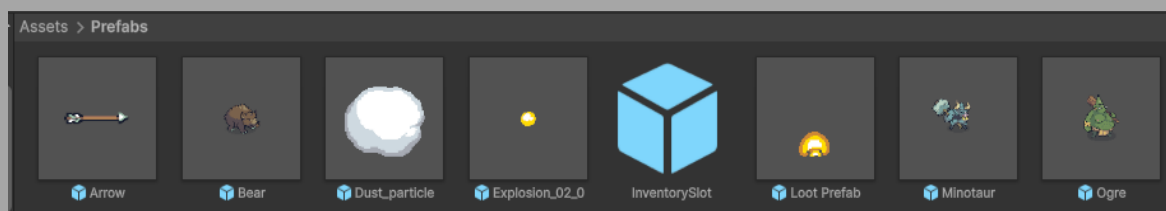
32. ábra: Ogre Animator



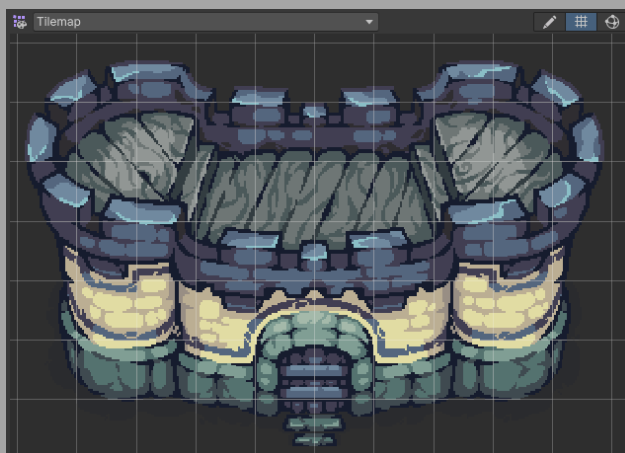
33. ábra: UI Canvas

```
public void PauseGame()  
{  
    isPaused = true;  
    Time.timeScale = 0;  
    pauseCanvas.SetActive(true);  
    if (pauseCanvasGroup != null)  
    {  
        pauseCanvasGroup.alpha = 1;  
        pauseCanvasGroup.blocksRaycasts = true;  
        pauseCanvasGroup.interactable = true;  
    }  
}
```

34. ábra: Time Management



35. ábra: Prefab System



36. ábra: Tilemap Renderer

3.3 Adatmodell leírása

A Unity motor által kínált ScriptableObject technológiát és az objektumorientált (OOP) adatszerkezeteket alkalmaztam. Ez a megközelítés lehetővé teszi a gyors adathozzáférést és a memória-hatékony tárolást.

3.3.1 ScriptableObject alapú adatmodell (ItemSO)

A játékban található tárgyak specifikációit a Unity ScriptableObject technológiájával valósítottam meg. Az ItemSO osztály szolgál adat-konténerként, amelyben az alábbi fix attribútumokat tárolom:

- **Alapadatok:** Tárgy neve (itemName), leírása (itemDescription) és a felhasználói felületen megjelenő ikonja (icon).
- **Logikai mezők:** Egy bool érték jelzi, ha a tárgy aranyként funkcionál (isGold), illetve meghatározható a maximális stack méret (stackSize).
- **Statisztikai módosítók:** A tárgyak közvetlen hatással vannak a játékmenetre, tárolva az életerő (currentHealth, maxHealth), sebesség (speed) és sebzés (damage) módosító értékeket.

3.3.2 Dinamikus adatszerkezetek

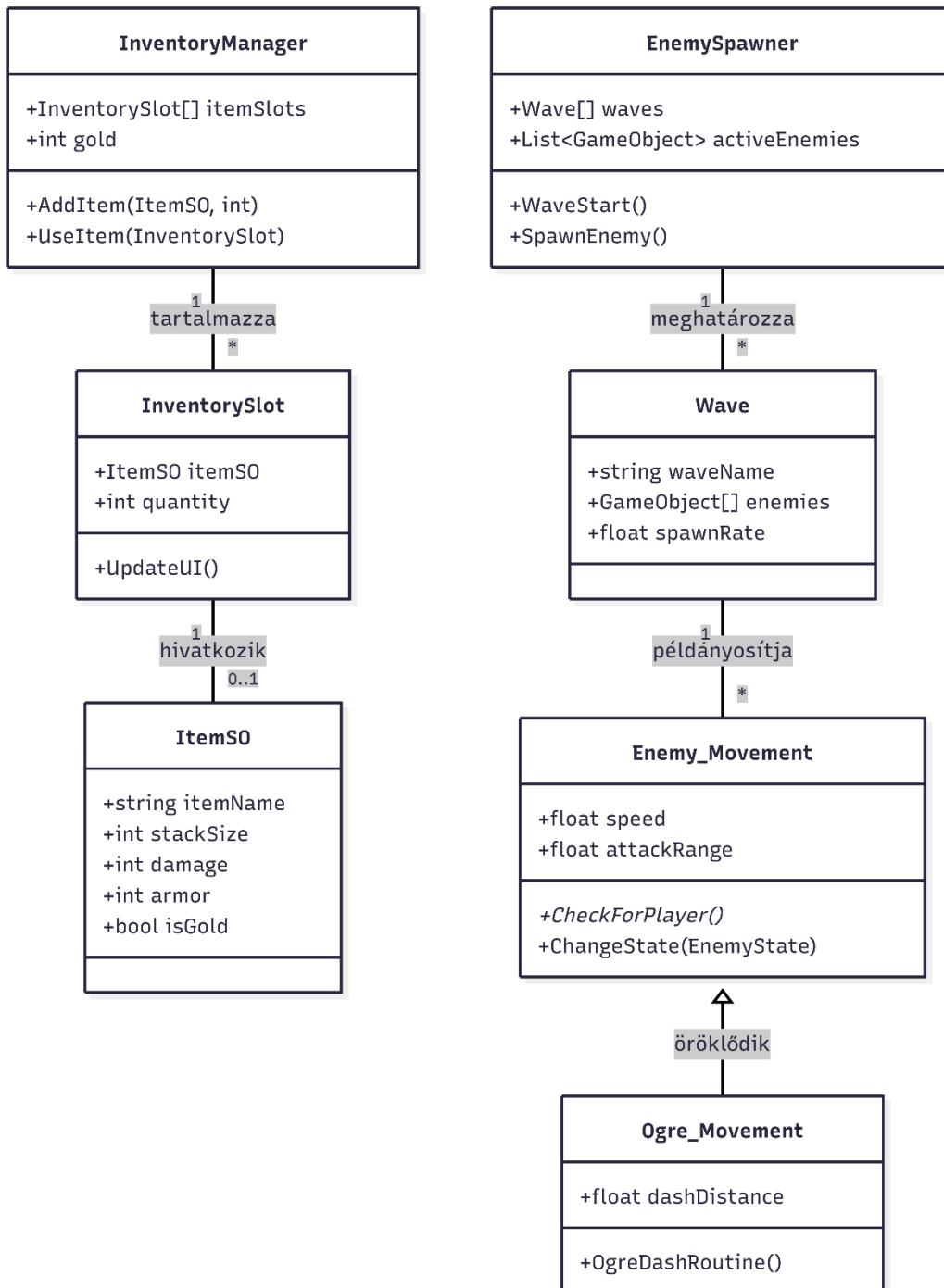
A játék futása közben változó adatokat központi menedzser osztályok kezelik, biztosítva az adatok folytonosságát.

- **Inventory Rendszer:** Az InventoryManager egy fix méretű InventorySlot[] tömböt használ a birtokolt tárgyak tárolására. Minden slot tartalmazza az adott ItemSO referenciáját és a mennyiséget (quantity).
- **Hullámkezelő (Wave) modell:** Az ellenségek hullámokban történő érkezését egy bővíthető Wave osztály definiálja az EnemySpawner-ben. Ez összefogja a hullám nevét, az ellenség-prefabbok tömbjét (enemies[]) és a megjelenési gyakoriságot.
 - A Hullám rendszer struktúrája:
 - **GameObject[] enemies:** Az adott hullámban megjelenő ellenség típusok.
 - **float spawnInterval:** Két ellenség megjelenése közötti idő.
 - **int totalEnemies:** A hullám teljes létszáma.
- **Runtime listák:** Az aktív ellenségek nyomon követésére egy dinamikus List<GameObject> listát használok, amelyből az egységek megsemmisülésekor automatikusan törölődnek a referenciák.

3.3.3 Osztályhierarchia és öröklődés

A fejlesztés során kihasználtam az objektumorientált programozás előnyeit. Az ellenségek mozgáslogikája egy hierarchikus struktúrára épül:

- **Enemy_Movement**: Ez az ősosztály tartalmazza az alapvető mesterséges intelligencia állapotokat (Idle, Chasing, Attacking) és a célpontkeresési logikát.
- **Ogre_Movement**: Az alaposztályból származik le, kiegészítve azt egy speciális, coroutine-alapú "Dash" képességgel és egyedi állapotkezeléssel.



3.4 Részletes feladatspecifikáció, megvalósítás

3.4.1 Főmenü (Main Menu) technikai megvalósítása

A főmenü felelős a szoftver indításáért, bezárásáért és a konfigurációk kezeléséért. A menü logikáját több, specializált szkript között osztottam fel.

- **MainMenuManager.cs:** A legalapvetőbb navigációért felelős komponens. Ez kezeli a játék elindítását a `SceneManager.LoadScene(1)` metódussal, valamint a programból való kilépést a `QuitGame()` függvényen keresztül.

```
public class MainMenuManager : MonoBehaviour
{
    0 references
    public void StartGame()
    {
        SceneManager.LoadScene(1);
    }

    0 references
    public void QuitGame()
    {
        Debug.Log("Kilépés...");
        Application.Quit();
    }
}
```

- **MainMenuOpenSettings.cs:** Ez a szkript vezérli a menüpanelek közötti váltást. Az `OpenSettings()` és `CloseSettings()` metódusok a `CanvasGroup` komponens segítségével (`alpha`, `interactable`, `blocksRaycasts`) teszik láthatóvá vagy láthatatlanná a beállítások ablakot, miközben elrejtik vagy megjelenítik a fő navigációs gombokat. Emellett ez a szkript indítja el a menüzenét és kezeli az ablaknyitáskor hallható hangeffektusokat is, melyet Bangó Máté valósított meg.

```
public void OpenSettings()
{
    settingsCanvas.SetActive(true);
    settingsCanvasGroup.alpha = 1;
    settingsCanvasGroup.blocksRaycasts = true;
    settingsCanvasGroup.interactable = true;
    startButton.SetActive(false);
    settingsButton.SetActive(false);
    quitButton.SetActive(false);

    AudioManager.instance.Play("PopUpOpen");
}

0 references
public void CloseSettings()
{
    settingsCanvas.SetActive(false);
    settingsCanvasGroup.alpha = 0;
    settingsCanvasGroup.blocksRaycasts = false;
    settingsCanvasGroup.interactable = false;
    startButton.SetActive(true);
    settingsButton.SetActive(true);
    quitButton.SetActive(true);

    AudioManager.instance.Play("PopUpClose");
}
```

- **SettingsMenuManager.cs:** A beállítások menü logikai központja.
 - Hangerőkezelés: A Master, Music és SFX csúszkák (Slider) értékeit logaritmikus skálán számítja át az AudioManager számára, biztosítva a fülnek természetes hangerőváltozást.
 - Adattárolás: A beállítások (hangerő szintek) a PlayerPrefs osztály segítségével kerülnek mentésre és betöltésre, így a szoftver megjegyzi a felhasználó preferenciáit a következő indításnál is.
 - Grafika: Kezeli a minőség beállítások váltását a beépített QualitySettings rendszeren keresztül.

```
1 reference
public void ChangeSFXVolume()
{
    float val = sfxVol.value;
    mainAudioMixer.SetFloat("SFXVol", Mathf.Log10(val) * 20);
    PlayerPrefs.SetFloat("SFXVol", val); // Mentés
}

0 references
public void ChangeGraphicsQuality()
{
    QualitySettings.SetQualityLevel(graphicsDropdown.value);
}
```

```
masterVol.value = PlayerPrefs.GetFloat("MasterVol", 0.75f);
musicVol.value = PlayerPrefs.GetFloat("MusicVol", 0.75f);
sfxVol.value = PlayerPrefs.GetFloat("SFXVol", 0.75f);
```

- **MainMenuParallax.cs:** Egy vizuális kényelmi funkció, amely az egér pozíciója (Input.mousePosition) alapján mozgatja el a háttérretegeket a megadott moveModifier értékkel. Ez mélységérzetet (parallax hatás) ad a főmenünek, a szebb megjelenés érdekében.

```
void Start()
{
    startPos = transform.localPosition;
}

Unity Message | 0 references
void Update()
{
    Vector2 pOffset = new Vector2(Input.mousePosition.x - Screen.width / 2, Input.mousePosition.y - Screen.height / 2);
    transform.localPosition = new Vector2(startPos.x + pOffset.x * moveModifier, startPos.y + pOffset.y * moveModifier);
}
```

3.4.2 Inventory és tárgykezelő rendszer

Adatvezérelt alapok (ScriptableObjects):

- A rendszer magját az **ItemSO.cs** alkotja. A Unity ScriptableObject osztályát kihasználva a tárgyak tulajdonságait nem példányonként tárolja a szoftver, hanem külső adatfájlokban. Ez jelentősen csökkenti a memóriefelhasználást és lehetővé teszi új tárgyak létrehozását kódolás nélkül.
 - ItemSO osztály változói (37. ábra) három fő kategóriába sorolhatóak:
 1. **Azonosító adatok** : Név, leírás és ikon a vizuális megjelenéshez
 2. **Rendszeradatok**: isGold logikai változó határozza meg a tárgy típusát (valuta vagy tárgy), a stackSize pedig a tárhelygazdálkodást szabályozza
 3. **Statisztikai módosítók**: Ezek az értékek közvetlenül a StatsManager.cs változóit befolyásolják, amit Bacskai Bence valósított meg.:

VÁLTOZÓ TÍPUSA	VÁLTOZÓ NEVE
STRING	itemName
STRING	itemDescription
SPRITE	icon
BOOL	isGold
INT	stackSize
STATISZTIKAI MÓDOSÍTÓK	
INT	currentHealth
INT	maxHealth
INT	speed
INT	damage
FLOAT	duration

37. ábra: ItemSO változói

A kód elején található „[CreateAssetMenu(fileName = "New Item", menuName = "ScriptableObjects/Item")]” sor egy speciális Unity attribútum, amely jelentősen megkönnyíti a fejlesztői munkamenetet.

Ennek előnyei a fejlesztés során:

- **Asset-alapú tartalomgyártás:** Lehetővé teszi, hogy a Unity Editorban a jobb egérgombbal kattintva, közvetlenül a projekt mappájában hozzák létre új tárgyakat, anélkül, hogy minden egyes tárgyhoz külön szkriptet vagy Prefabot kellene készítenem. (38. ábra)
- **Központi adatmódosítás:** Ha megváltoztatom egy ItemSO fájlban a sebzés vagy a gyógyítás értékét, az a játék összes pontján (a boltban, az inventory-ban) azonnal frissül.

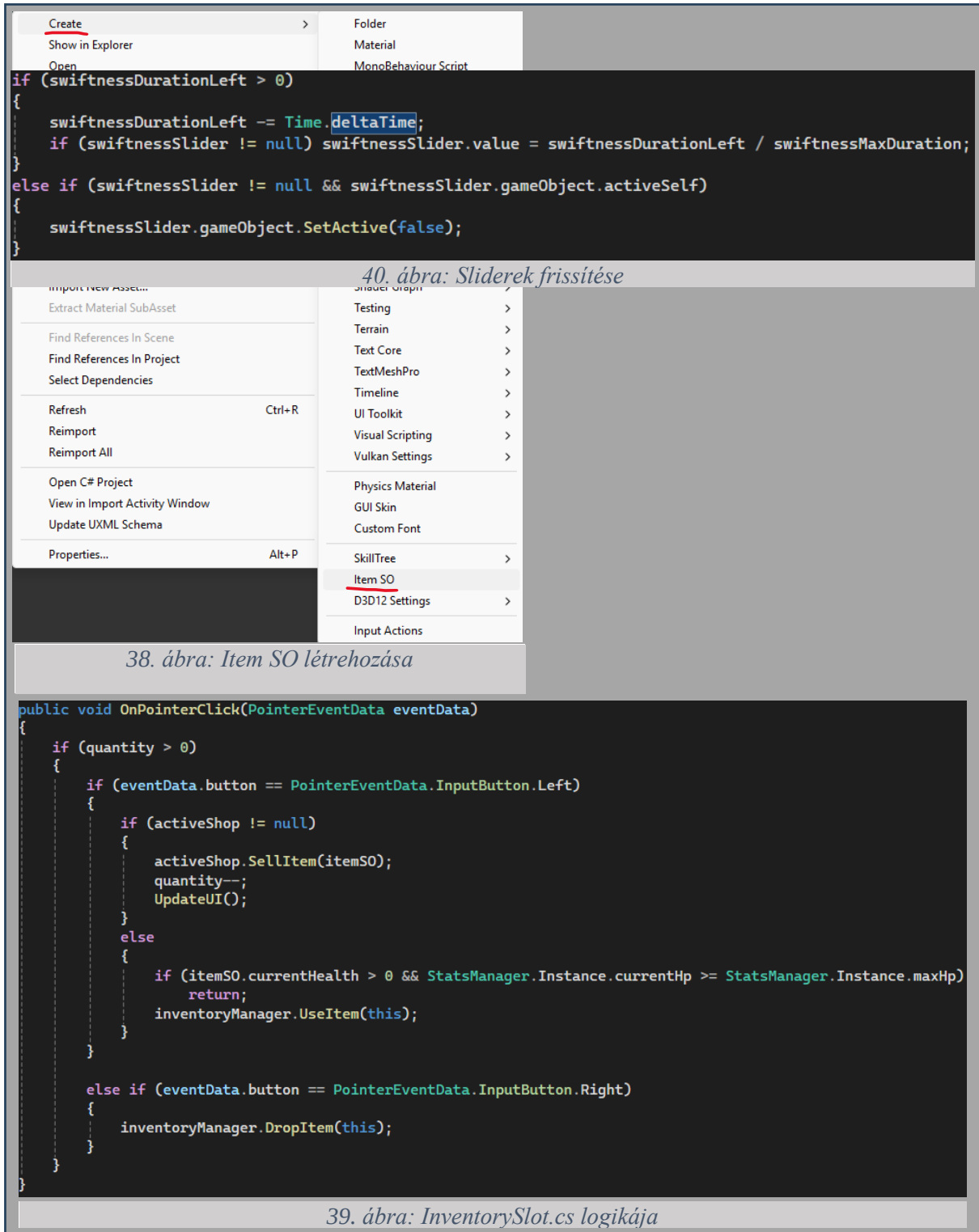
Az Inventory rendszer főbb komponensei és mechanizmusai:

- **InventoryManager.cs:** A rendszer központi vezérlője. Ez kezeli az arany mennyiségét, a tárgyak hozzáadását (AddItem) és a loot felvételét. Tartalmazza a tárhely slotok listáját, és felelős azért, hogy új tárgy felvételekor az az első szabad helyre vagy egy már meglévő stacket (halmot) növelve kerüljön be.
- **InventorySlot.cs:** Az egyes cellák logikájáért felelős. Implementálja az IPointerClickHandler interfészt, így képes kezelni a jobb és bal egérgombos kattintásokat (39. ábra):
 - Bal klikk: Tárgy használata vagy eladása, ha a bolt nyitva van.
 - Jobb klikk: Tárgy eldobása a világba.
- **Loot.cs:** A játéktérben megjelenő, felvehető tárgyak reprezentációja. Eseményalapú programozást használ (OnItemLooted event), így amikor a játékos hozzáér a tárgyhoz, a rendszer egy jelzést küld az inventory-nak a felvételtől, majd megsemmisíti a tárgyat a szinterről.
- **UseItem.cs:** A bájitalok logikai lekezelője. Ez a szkript felelős a StatsManager értékeinek módosításáért, amit Bacscai Bence valósított meg.
 - Időalapú effektek: Coroutine-okat (EffectTimer) használ a temporális erősítések kezelésére, amelyek a megadott idő lejártá után automatikusan levonják a bónusz értékeket.
 - Vizuális visszajelzés: Itt történik a korábban említett UI sliderek (csúszkák) frissítése, amelyek valós időben mutatják a bájitalok hátralévő hatóidejét. (40. ábra)

Technikai megoldások:

- A tárgyak használatakor a szoftver ellenőrzi a játékos állapotát (például teli életerő esetén nem engedi elhasználni a gyógyító italt), ami megelőzi az erőforrások felesleges pazarlását.

3.4.3 Inventory Ábrák



3.4.4 Kereskedelmi rendszer (Shop) technika felépítése

A bolt mechanikája lehetővé teszi a játékos számára, az összegyűjtött arany beváltását fejlesztésekre és fogyasztható tárgyakra.

A rendszer három fő pillérre épül:

- az eladóra (interakció) => **Shopkeeper.cs**,
- a boltvezérlőre (logika) => **ShopManager.cs**
- és a kínálati elemekre (adatmegjelenítés) => **ShopSlot.cs**

A rendszer komponensei:

- **Shopkeeper.cs** (Az interakció kezelője) (40. ábra):
 - Ez a szkript felelős a bolt fizikai jelenlétéért és a felhasználói felület (UI) aktiválásáért.
 - **Trigger-alapú észlelés:** Az `OnTriggerEnter2D` és `OnTriggerExit2D` metódusok figyelik, ha a lovag az eladó közelébe ér. Ekkor egy animáció jelzi az interakció lehetőségét.
 - **Állapotkezelés:** A bolt megnyitásakor (`OpenShop`) a játék megáll (`Time.timeScale = 0`), biztosítva a nyugodt vásárlást. A szkript `Static Event`-et (`OnShopStateChanged`) használ, hogy értesítse a rendszer többi részét (pl. az inventory-t) a bolt állapotáról.
- **ShopManager.cs** (A tranzakciós logika) (41. ábra):
 - Ez a komponens végzi a tényleges számításokat és ellenőrzéseket.
 - Vásárlási folyamat (`TryBuyItem`): Mielőtt a tárgy bekerülne az inventory-ba, a rendszer három feltételt ellenőriz:
 - a tárgy létezését,
 - a játékos aranyának mennyiségét,
 - és hogy van-e szabad hely a táskában (`HasSpaceForItem`).
 - Dinamikus kínálat: A `PopulateShopItems` metódus teszi lehetővé, hogy a bolt kínálata rugalmasan változzon (pl. külön fül a fegyvereknek és a potionöknek).
 - Eladás: Kezeli a tárgyak visszaváltását, ahol a `ShopSlot`-ban tárolt ár alapján növeli a játékos aranyát.

- **ShopSlot.cs** (A kínálati elem):
 - A bolt felületén megjelenő egyedi cellákért felelős. Implementálja a Unity `EventSystem` interfészeit (`IPointerEnterHandler`, `IPointerExitHandler`), így amikor a játékos az egérrel egy tárgy fölé mutat, megjelenik egy információs ablak (`ShopInfo`) a tárgy részletes leírásával.

Technikai megoldások és optimalizálás:

1. UI Animációk: A bolt megnyitása és bezárása nem azonnali, hanem a `shopUIAnimator` és `Coroutine`-ok segítségével animált, ami simább felhasználói élményt nyújt.
2. Referencia-kezelés: A bolt és az inventory közötti kommunikáció direkt hivatkozásokon és eseményeken keresztül zajlik, így a tranzakciók során az arany mennyisége és a slotok tartalma azonnal, hiba nélkül frissül.
3. Vizuális visszajelzés: A konzol üzenetek (`Debug.Log`) segítik a fejlesztőt a tranzakciók nyomon követésében (pl. "Not enough gold" hibaüzenetek), amiket a végleges verzióban UI feliratok válthatnak fel
4. `static event Action<ShopManager, bool> OnShopStateChanged`, használata miatt, az Inventory tudni fogja a bolt helyzetét, így tudja a szoftver, hogy mikor van eladási vagy tárgy használati módban.

Kereskedelmi rendszer (Shop) technika ábrái

```
private IEnumerator ToggleShopRoutine()
{
    isAnimating = true;

    if (!isShopOpen)
    {
        isShopOpen = true;
        UIManager.OpenWindowCount++;

        Time.timeScale = 0;

        AudioManager.instance.Play("PopUpOpen");

        OnShopStateChanged?.Invoke(shopManager, true);
        shopCanvasGroup.alpha = 1;
        shopCanvasGroup.blocksRaycasts = true;
        shopCanvasGroup.interactable = true;
        OpenPotionShop();

        if (shopUIAnimator != null)
        {
            shopUIAnimator.SetTrigger("slidein");
        }

        yield return new WaitForSecondsRealtime(animationTime);
    }
    else
    {

```

40. ábra: Belépés a Shop-ba

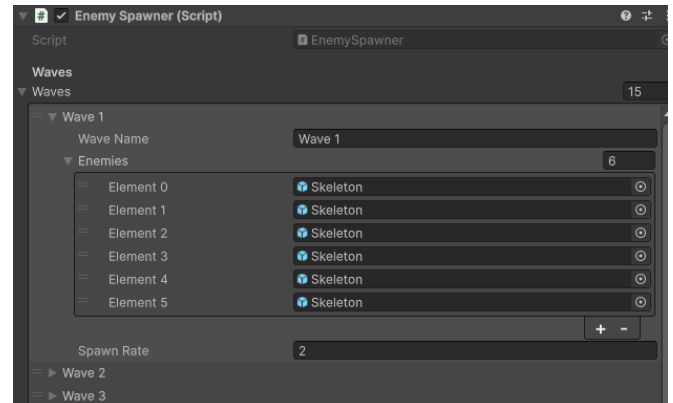
```
public void TryBuyItem(ItemSO itemSO, int price)
{
    Debug.Log($"Trying to buy: {itemSO.itemName}, Price: {price}");
    if (itemSO == null)
    {
        return;
    }
    if (inventoryManager.gold < price)
    {
        Debug.Log("Not enough gold");
        return;
    }
    if (!HasSpaceForItem(itemSO))
    {
        Debug.Log("Not enough slots in inventory");
        return;
    }
    Debug.Log("Successful buy");
    inventoryManager.gold -= price;
    inventoryManager.goldText.text = inventoryManager.gold.ToString();
    inventoryManager.AddItem(itemSO, 1);
}
```

41. ábra: Tárgy vásárlási logika

3.4.5 Játékmenet és ellenségkezelés

EnemySpawner.cs (Hullám-kezelő):

- Ez a class ügyel a játék fő funkciójára, az ellenség hullámokra. Egy egyedi Wave osztályt használ, amelyben definiálható a hullám neve, az ellenségek listája és a megjelenési sebesség (42. ábra).
 - Figyeli az aktív ellenségek listáját (activeEnemies). Ha a lista kiürül, és minden ellenség legyártásra került, megnyitja a felkészülési fázist és megjeleníti a „Start the Wave” gombot.
 - Ha a 15. hullám után sem marad ellenség, aktiválja a WinCanvas-t, mely tudatja a játékosnak, hogy kijátszotta a játékot.



42. ábra: Wave kezelő a Unity-ben

Ogre_Movement.cs (Főellenség kiegészítő script):

- Az alap ellenség-mozgásból öröklődött de kiegészíti azt egy komplexebb támadási logikával.
- Dash mechanika (43. ábra):
 - Coroutine-t használ a rohamozáshoz, amely során a linearVelocity módosításával hirtelen felgyorsítja a karaktert a játékos irányába, miközben sebést és hátralökést okoz.

```
IEnumerator OgreDashRoutine(Transform target)
{
    isDashing = true;
    dashCooldownTimer = dashCooldown;

    ChangeState(EnemyState.Dash);

    Vector2 dashDir = (target.position - transform.position).normalized;

    float startTime = Time.time;
    while (Time.time < startTime + dashDuration)
    {
        GetComponent<Rigidbody2D>().linearVelocity = dashDir * dashSpeed;
        yield return null;
    }

    GetComponent<Rigidbody2D>().linearVelocity = Vector2.zero;
    isDashing = false;
    ChangeState(EnemyState.Chasing);
}
```

43. ábra: Dash Coroutine

3.4.6 Vár élete és újraéledési rendszer

Base_Health.cs (Várkapu védelme):

A vár életterejét kezelő script. Legfontosabb feladata a TakeDamage metódus, amely ha eléri a nullát, azonnal megszakítja a játékot és aktiválja a GameOverCanvas-t.

```
public void TakeDamage(int amount)
{
    currentHp -= amount;

    if (currentHp < 0) currentHp = 0;

    UpdateBaseUI();

    if (currentHp <= 0)
    {
        GameOver();
    }
}
```

```
void GameOver()
{
    dieCanvas.SetActive(false);
    dieCanvasGroup.alpha = 0;

    GameOverCanvas.SetActive(true);
    GameOverCanvasGroup.alpha = 1;
    GameOverCanvasGroup.blocksRaycasts = true;
    GameOverCanvasGroup.interactable = true;

    Time.timeScale = 0;
}
```

RespawnManager.cs (Újraéledési rendszer): A játékos halálakor egy Coroutine-t indít, amely visszaszámlálást jelenít meg a képernyőn, majd a megadott helyen újra aktiválja a játékos objektumát. Ez biztosítja, hogy a játék ne érjen véget a karakter halálával.

```
IEnumerator RespawnRoutine(GameObject player, float delay, Vector3 spawnPos)
{
    player.SetActive(false);

    if (dieCanvas != null) dieCanvas.SetActive(true);
    if (dieCanvasGroup != null) dieCanvasGroup.alpha = 0;

    float remainingTime = delay;

    while (remainingTime > 0)
    {
        if (dieCanvasGroup != null && dieCanvasGroup.alpha < 1)
        {
            dieCanvasGroup.alpha += Time.deltaTime * 2f;
        }

        if (timerText != null)
        {
            timerText.text = "Respawn: " + remainingTime.ToString("f1") + "s";
        }

        remainingTime -= Time.deltaTime;
        yield return null;
    }

    if (dieCanvasGroup != null) dieCanvasGroup.alpha = 0;
    if (dieCanvas != null) dieCanvas.SetActive(false);
    if (timerText != null) timerText.text = "";

    player.transform.position = spawnPos;
    Player_Movement pm = player.GetComponent<Player_Movement>();
    if (pm != null) pm.ResetMovement();
    StatsManager.Instance.currentHp = StatsManager.Instance.maxHp;
    player.SetActive(true);
}
```

3.4.7 Rendszerszintű vezérlők

UIManager.cs (Globális ablakkezelő): Egy static osztály, amely egy központi számlálót (OpenWindowCount) vezet az éppen nyitott UI panelekről. Ez elengedhetetlen a bemeneti konfliktusok elkerüléséhez (például ne lehessen megnyitni a Karakterstatisztika menüt, miközben a Bolt nyitva van).

PauseManager.cs (Szünet és navigáció):

- A játék irányítása során az „Escape” billentyű intelligens, kontextusfüggő feladatkört lát el. A szoftver a leütés pillanatában megvizsgálja a játék aktuális állapotát, és ennek megfelelően dönt a végrehajtandó akcióról:
 1. Aktív ablakok bezárása: Ha a játékos éppen egy interaktív felületet használ (például a Boltot vagy a Képességfát) az „Escape” billentyű elsődleges funkciója az adott ablak bezárása és a játékmenethez való visszatérés. Ez a mechanizmus biztosítja, hogy a felhasználó gyorsan, egyetlen gombbal kiléphessen a menükből.
 2. Szüneteltetés (Pause): Amennyiben nincs megnyitva egyetlen funkcionális ablak sem, az „Escape” billentyű elindítja a játék szüneteltetését, megjelenítve a Pause Menüt. Ekkor a Time.timeScale értéke 0-ra módosul, megállítva a játékidőt.

Technikai megvalósítás:

A fejlesztés során ezt a logikát a UIManager.cs – ben lévő „IsAnyUIOpen” globális változóval és a PauseManager szkript közötti kommunikációval valósítottam meg. A szoftver egy feltételvizsgálattal (if-else szerkezet) dönti el, hogy a menü bezárása vagy a globális szüneteltetés metódusát hívja-e meg. Ez az eljárás megakadályozza a grafikai felületek egymásra csúszását.

```
if (Input.GetButtonDown("Pause")) //ESC
{
    if (settingsCanvas != null && settingsCanvas.activeSelf)
    {
        CloseSettings();
        return;
    }

    if (skillTree != null && skillTree.skillTreeOpen)
    {
        skillTree.SkillTreeLeave();
        return;
    }

    if (statsUI != null && statsUI.statsOpen)
    {
        statsUI.CloseStats();
        return;
    }

    if (shopKeeper != null && shopKeeper.isShopOpen)
    {
        shopKeeper.CloseShop();
        return;
    }

    if (isPaused) ResumeGame();
    else if (!UIManager.IsAnyUIOpen) PauseGame();
}
```

3.4.8 Vizuális effekt és optimalizálás

ScreenFlashEffect.cs:

- A Universal Render Pipeline (URP) Vignette effektjét használja. Ha a játékos sérül, egy vörös villanást, ha a pajzsa törik, egy szürke villanást generál. A hatást Coroutine-ok segítségével fokozatosan halványítja el, professzionális harci élményt nyújtva.

```
IEnumerator FlashRoutine()
{
    volume.priority = 100;
    vignette.intensity.value = flashIntensity;

    yield return new WaitForSeconds(0.06f);

    while (vignette.intensity.value > 0)
    {
        vignette.intensity.value -= Time.deltaTime * fadeSpeed;

        if (vignette.intensity.value < 0) vignette.intensity.value = 0;

        yield return null;
    }
    volume.priority = 1;
}

1 reference
IEnumerator ShieldFlashRoutine()
{
    shieldVolume.priority = 101;
    shieldVignette.intensity.value = flashIntensity;

    yield return new WaitForSeconds(0.1f);

    while (shieldVignette.intensity.value > 0)
    {
        shieldVignette.intensity.value -= Time.deltaTime * fadeSpeed;

        if (shieldVignette.intensity.value < 0) shieldVignette.intensity.value = 0;

        yield return null;
    }
    shieldVolume.priority = 1;
}
```

AutoDestroyParticle.cs (Memória optimalizálás):

- A vizuális effekt (füst) példányosítása után a megadott idő elteltével törli azokat a memóriából, megelőzve a játék lassulását.

```
public class AutoDestroyParticle : MonoBehaviour
{
    public float destroyDelay = 1f;

    Unity Message | 0 references
    void Start()
    {
        Destroy(gameObject, destroyDelay);
    }
}
```

3.4.9 Tesztesetek

A szoftver megbízhatóságának ellenőrzése érdekében az alábbi kritikus teszteseteket hajtottam végre:

Kontextusfüggő Esc kezelés

Lépések:

1. Bolt megnyitása.
2. Escape billentyű megnyomása.
3. Escape billentyű ismételt megnyomása.

Elvárt eredmény:

Az első Escape billentyű megnyomásával a bolt bezárása, a második megnyomással a Pause menü megnyitása.

Eredmény:

Sikeres, a Pause menü akkor nyílik meg, ha a bolt nincsen megnyitva.

Vásárlási logika és aranykezelés

Lépések:

1. Meglévő arany ellenőrzése (kevesebbnek kell lennie, mint a tárgy).
2. Vásárlás megpróbálása.

Elvárt eredmény:

Rendszer hibaüzenet küldése („Not enough gold”), az arany nem elvétele, és a tárgy ki nem adása.

Eredmény:

Sikeres, a játékos akkor tud vásárolni a boltban, ha elég arannyal rendelkezik.

Tárgyhasználat és Buff időzítő

Lépések:

1. Hatóidővel rendelkező bájjal elhasználata.
2. Hatóidő lejáratának megvárása

Elvárt eredmény:

Visszaszámláló csúszka megjelenése, hatóidő lejáratával, a lovag tulajdonságainak vissza állítása.

Eredmény:

Sikeres, megjelenik a csúszka, amint lejár a hatóidő, a játékostól elveszi a megkapott erősítést.

3.4.10 Hibák felsorolása (bugok)

Falon való átcúszás:

Amikor a játékos szorosan egy fal mellett áll és egy ellenség hátralöki a játékost előfordul, hogy a játékos átmegy a falon, és innentől kezdve bármelyik falon át tud menni a lovag.

Ogre és a kapu jelenség:

A 15. hullám végén megjelenő főellenség (Ogre) legyőzése nélkül is elérhető volt a győzelem. Ha a játékos hagyta, hogy az Ogre sebzést okozzon a várra, a többi ellenséghez hasonlóan, megsemmisül. Amennyiben a vár életerejéje nem fogy el teljesen, a rendszer aktiválja a győzelmi képernyőt, mivel ez volt a legutolsó ellenség a 15. hullámból.

Megoldás:

Az Ogre a vár maximális életerejét, tehát 5 egységnyi sebzést okoz a várnak, mellyel egyből ledönti a várfalat, és a játék elvesztését garantálja.

UI Konfliktus, menü egymásranyítás

A fejlesztés korai szakaszában a különböző kezelőfelületek (Bolt, Képességfa) függetlenül működtek egymástól. Ez lehetővé tette, hogy a játékos a már megnyitott Bolt ablakra rányissa a Pause menüt, vagy fordítva.

A hiba következményei:

- Logikai hiba: A játékos el tudott navigálni a főmenübe vagy szüneteltetni tudta a játékot olyan állapotban, amikor a Bolt szkriptje még aktív tranzakciót várt.
- Bemeneti konfliktus: Az egérekattintások és a billentyűleütések (pl. Esc) egyszerre próbálták irányítani mindkét ablakot.
- Vizuális hiba: A grafikai elemek egymásra csúsztak, olvashatatlanná téve a kezelőfelületet.

Megoldás:

A hiba kijavítására bevezettem egy központi „UIManager” osztályt, amely egy statikus számlálóval (OpenWindowCount) figyeli a nyitott ablakok számát.

- Mostantól minden menü nyitása előtt a szoftver ellenőrzi az „IsAnyUIOpen” állapotot.
- Ha egy ablak már nyitva van, az „Escape” gomb, az aktuális ablakot zárja be.

3.4.11 Továbbfejlesztési lehetőségek

- **Többszintű pályarendszer (Map Progression)**

A jelenlegi egypályás játékot további pályákkal lehetne kibővíteni.

Megvalósítás:

A 15. hullám után a játékos nem egy győzelmi képernyőt kapna, hanem lehetőséget a következő helyszínre (pl. Barlang, Alvilág) való utazásra. Ehhez új Tilemap rétegek és helyszínspecifikus vizuális effektek (pl. köd, fényhatások) készítése/keresése szükséges.

- **Bővített ellenségtípusok (Távolsági támadók)**

A jelenlegi közelharc egységek mellé távolsági ellenségek kerülnének a játékba.

Megvalósítás: Egy új AI-logika bevezetése, amely nem a játékos közvetlen közelébe navigál, hanem egy meghatározott távolságban megáll, és lövedékeket indít.

- **Különféle lovag karakterek**

A játékos választhatna különböző tulajdonságokkal rendelkező lovagok közül az indításkor.

Megvalósítás: Az inventory-nál már használt ScriptableObject technológiát alkalmaznám a karaktereknél is (CharacterSO).

- Nehézlovas: Lassabb mozgás, de magasabb kezdő életerő és páncélzat.
- Orgyilkos: Alacsony védelem, de magas kritikus sebzés és gyorsabb mozgás.
- Varázsló-lovag: Kisebb kard-sebzés, de gyorsabb képességhasználat.

4. Összegzés

A záródolgozatom elkészítése jóval több volt számomra egy egyszerű programozási feladatnál. Ez volt az első olyan komplex projektem, ahol a tervezéstől a hibajavításig minden folyamatot magamnak kellett elvégeznem.

Szakmai tapasztalatok és kompetenciák

A munka során a legnagyobb előrelépést az objektumorientált szemléletmódban érzem. Még korábban csak elszigetelt kódokat írtam, itt megértettem, hogyan építhető fel egy olyan rendszer, ahol az elemek (mint a tárgyak) központi adatfájlokból (ScriptableObject) dolgoznak. Ez nemcsak átláthatóbbá tette a projektet, de megmutatta azt is, hogyan lehet folyamatosan bővíthető szoftvert fejleszteni. Emellett sokat tanultam a Unity belső működéséről, ami megmutatta, hogy milyen nehézségekkel jár egy játék készítése.

A fejlesztés nehézségei

Nem ment minden zökkenőmentesen, a legnehezebb része a különböző rendszerek (inventory, bolt) közötti kommunikáció volt. Meg kellett tanulnom, hogyan akadályozzam meg a logikai ütközéseket, például hogy ne nyíljanak egymásra a menük. Amint előrehaladtam a fejlesztés során, mindig előjött egy apró hiba, amit orvosolni kellett, így az általam elképzelt idő amit rá akartam szánni a játék egy részére, mindig kétszer annyi volt.

Jövőbeli célok

A projekt elkészítése alatt izgatottan vártam, hogy milyen is lesz ez a játék, és örömet okozott amikor elkészítettem a játék egy részét, mellyel egyre és egyre jobb lett a játékelmény. A jövőben biztos vagyok benne, hogy kipróbálom magamat még egyszer ebben a térben, de az sem kizárható, hogy ezzel foglalkoznék.

5. Ábrajegyzék

1. ábra: A One Knight Army főmenüje	10
2. ábra: A One Knight Army logója	10
3. ábra: Start Game gomb	11
4. ábra: Settings gomb	11
5. ábra: Exit gomb	11
6. ábra: Settings menü	12
7. ábra: Grafikai menü	13
8. ábra: Karakter mozgása	14
9. ábra: Lovag futás	14
10. ábra: Lovag csapás	15
12. ábra: Lovag védekezés	15
11. ábra: Lovag íjászat és lövés	15
13. ábra: Kereskedő NPC	16
14. ábra: Ikon visszajelzés	16
15. ábra: Bolt felület	17
18. ábra: Arany számláló	19
16. ábra: Lovag életerő	19
17. ábra: Vár életerő	19
19. ábra: Inventory	19
20. ábra: Inventory feltöltve	19
21. ábra: Tapasztalati pont	19
22. ábra: Időalapú visszajelzők	19
23. ábra: Karakterstatisztika menü	20
24. ábra: Képességfa	21
25. ábra: Szünet menü	23
29. ábra: Ellenség számláló	24
28. ábra: Hullám indikátor	24
27. ábra: Hullám indító gomb	24
26. ábra: Felkészülés	24
30. ábra: Ogre képessége, Rohamozás	26
29. ábra: Ogre, végső ellenség	26
31. ábra: Shop System Feladatkörei	30

35. ábra: Prefab System.....	30
34. ábra: Time Management.....	30
33. ábra: UI Canvas	30
32. ábra: Ogre Animator.....	30
36. ábra: Tilemap Renderer	30
37. ábra: ItemSO változói.....	35
40. ábra: Sliderek frissítése.....	37
39. ábra: InventorySlot.cs logikája	37
38. ábra: Item SO létrehozása.....	37
40. ábra: Belépés a Shop-ba	40
41. ábra: Tárgy vásárlási logika	40
42. ábra: Wave kezelő a Unity-ben	41
43. ábra: Dash Coroutine.....	41

6. Irodalomjegyzék, forrásmegjelölés

- Night Run Studio YouTube csatorna
 - elérhetőség: <https://www.youtube.com/@NightRunStudio>
- Raycastly YouTube csatorna (Főmenü és zenék beállítása)
 - elérhetőség: <https://www.youtube.com/@raycastly>
- AIA YouTube csatorna (általános segítség)
 - elérhetőség: <https://www.youtube.com/@AIAdev>
- Unity hivatalos tutorial sorozata
 - weboldal: <https://learn.unity.com/tutorial/start-learning-unity>
- Unity Asset Store:
 - weboldal: <https://assetstore.unity.com/>
- Itch.io
 - weboldal: <https://itch.io/>
- Tiny Swords Asset Pack
 - weboldal: <https://pixelfrog-assets.itch.io/tiny-swords>
- Saját GitHub Repository:
 - <https://github.com/matepiee/one-knight-army>